

Внутренние формы представления программы. Оптимизация программ

Теория автоматов и формальных языков

Лекция №12

Структура лекции

- Внутренние формы представления программ.
- Польская форма записи.
- Запись циклов и выражений.
- Тетрады.
- Виды оптимизации программ.
- Машинно-независимая оптимизация.

Внутренние формы представления программы

- Внутренняя форма — это промежуточная форма представления исходной программы, пригодная
 - для простой машинной обработки;
 - для интерпретации.

Польская форма записи

- Пусть операнды выбираются не из входного потока, а из специально отведенного для этого стека.
- Как и раньше в R хранится текущий считываемый символ.
- Если R является идентификатором или константой, то его значение заносится в стек и осуществляется считывание следующего символа (*правило "<операнд>::=идентификатор"*).
- Если R – бинарный оператор, то он применяется к двум верхним операндам в стеке, и затем они меняются на полученный результат (*правило "<операнд>::=<операнд><операнд><оператор>"*).
- Если R – унарный оператор, то он применяется к верхнему символу в стеке, который затем заменяется на полученный результат (*правило "<операнд>::=<операнд><оператор>"*).

Алгоритм польской формы записи

```
R := getchar
while R != # do
  case R of
    'идентификатор':  push(R)
    'бин.оператор':   pop(B), pop(A), Tmp := R(A,B), push(Tmp)
    'унарн.оператор': pop(A), Tmp := R(A), push(Tmp)
  endcase
  R := getchar
endwhile
```

Расширения польской формы записи

Безусловные переходы

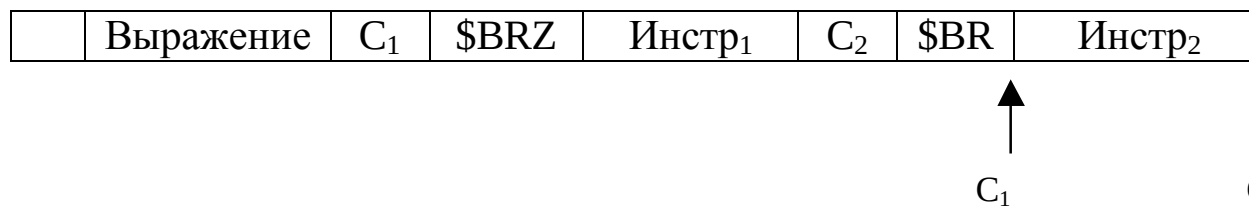
- Безусловный переход на метку (аналог GOTO L)
 - $L \text{ \$BRL (Branch to label)}$

Условные переходы

- $\langle \text{операнд1} \rangle \langle \text{операнд2} \rangle \text{ \$BRxZ / \$BRM}$
переход по значению ($<0, =0, >0$)

Условные конструкции

- $IF \langle \text{выражение} \rangle THEN \langle \text{инстр1} \rangle ELSE \langle \text{инстр2} \rangle$
- В ПФ мы имеем:
 - $\langle \text{выражение} \rangle \langle c_1 \rangle \text{ \$BRZ } \langle \text{инстр}_1 \rangle \langle c_2 \rangle \text{ \$BR } \langle \text{инстр}_2 \rangle$



Расширения польской формы записи

Описание массивов

- Пусть дано описание массива
 - ***ARRAY A[L1..U1,...,Lk..Uk]***
- в ПФ оно представляется в виде
 - ***L1 U1 ... Lk Uk A \$ADEC,***

Обращение к элементу массива и вызовы подпрограмм

- Конструкция $A[\langle \text{выр} \rangle, \dots, \langle \text{выр} \rangle]$ записывается в виде
 - $\langle \text{выр} \rangle \dots \langle \text{выр} \rangle A \$SUBS$

Расширения польской формы записи

- **Вызов функции**

вызов функции вида $f(x_1, x_2)$ в польской форме записи будет выглядеть как **$x_1 x_2 f \$CALL$** ,

- где x_1 и x_2 – аргументы, f – имя функции, $\$CALL$ – команда передачи управления по адресу, определяемому именем функции.

Расширения польской формы записи

Циклы

- ***FOR I=N1 TO N2 DO Operator***

может быть сконструирован на исходном языке:

I := N1;

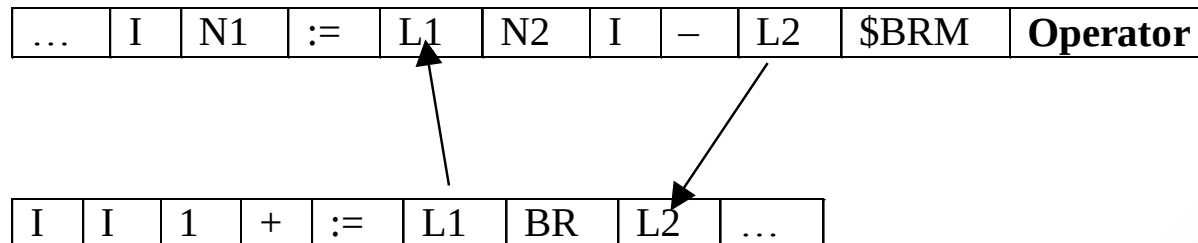
L1: IF I>N2 THEN GOTO L2;

Operator;

I:=I+1;

GOTO L1;

L2:



Циклы

BEGIN

INTEGER K;

ARRAY A[1..I-J];

K := 0;

L: IF I>J THEN

*K := K+A[I-J]*6*

ELSE

BEGIN

I := I+1;

I := I+1;

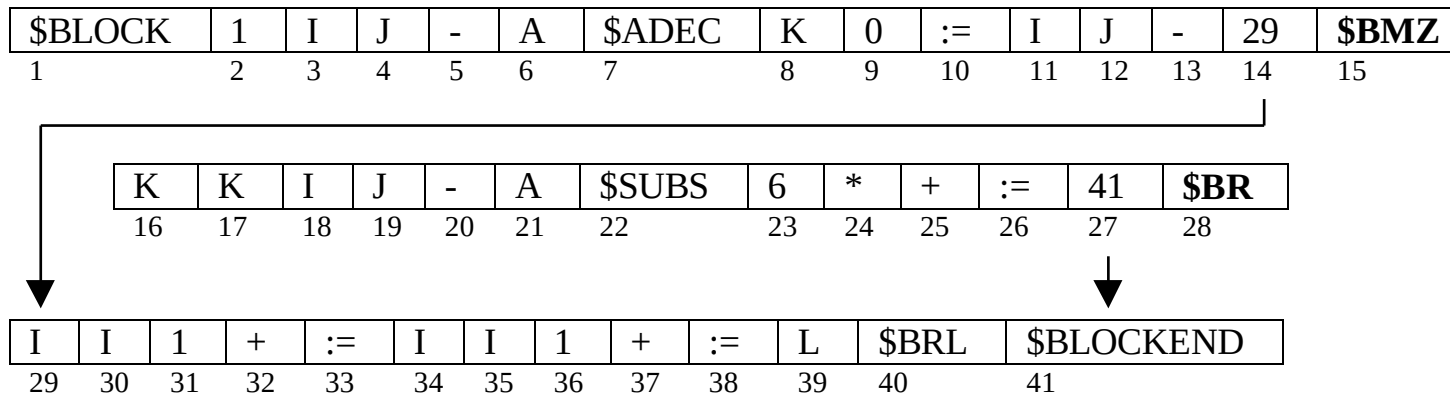
GOTO L;

END

END.

Реализация цикла

- Операторы блоков
- Польская форма записи фрагмента программы:



Хранение данных

- Хранение данных в польской форме записи выполняется в виде чисел и индексов таблицы имен.
- В *стеке аргументов* могут храниться данные разных типов.
- Одним из возможных вариантов организации рабочего стека является использование т.н. *тегового* стека.

Тетрады

Тот же фрагмент программы, представленный в форме тетрад, выглядит так:

(1) \$BLOCK	(10) + K, T4, T5
(2) - I, J, T1	(11) := T5,, K
(3) \$BOUNDS 1, T1	(12) \$BR 18
(4) \$ADEC A	(13) + I, 1, T6
(5) := 0,, K	(14) := T6,, I
(6) - I, J, T2	(15) + I, 1, T7
(7) \$BMZ 13, T2	(16) := T7,, I
(8) - I, J, T3	(17) \$BRL L
(9) * A[T3], 6, T4	(18) \$BLOCKEND

Тетрады

- Используемые операторы:

Оператор	Операнды	Описание
\$BR	i	переход на i-ю тетраду
\$BZ (BP,BM)	i, P	переход на i-ю тетраду, если P=0 (P>0,P<0)
\$BG (BL,BE)	i, P1, P2	переход на i-ю тетраду, если P1>P2 (P1<P2,P1=P2)
\$BRL	P	переход на тетраду, номер которой задан в P-м элементе таблицы символов
\$BOUNDS	P1, P2	P1 и P2 описывают граничную пару массива
\$ADEC	P	Массив описан в P. Если размерность массива равна n, то этой тетраде должны предшествовать n операторов \$BOUNDS, задающих n граничных пар

Работа с массивами

Введем пару операторов типа

- \$AINDX I
- \$AGET A, R

Здесь \$AINDX заносит аргумент I (индекс массива) в специальный стек – стек индексов.

Выражение "**A[1,2]+B[J]**" будет представлено в виде

- \$AINDX 1
- \$AINDX 2
- \$AGET A, T1
- \$AINDX J
- \$AGET B, T2
- + T1, T2, T3

Операторы и выражения

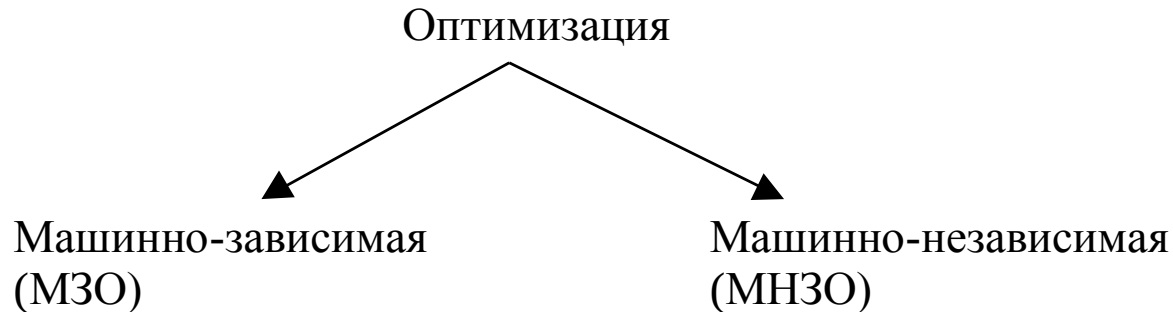
- Общий синтаксис языка
- Надо быть предельно внимательным при определении таких основных синтаксических категорий, как ***программа, оператор и выражение.***
- Пустой оператор.
- Разделитель ; перед закрывающей скобкой.

```
void main(void)
{ int a,b;

  3-4*b;
  1+2;

  for(2+a;1;)
      b+1;
}
```

Оптимизация программ



- МЗО связана с типом генерируемых команд и включается в фазу генерации кода (т.е. оптимизации подлежат машинные коды).
- МНЗО – отдельная фаза компилятора, предшествующая генерации кода. Она включается на этапе генерации промежуточного кода – внутренней формы представления программы.

Виды машинно-независимой оптимизации

Оптимизации подлежат:

- время выполнения;
- емкостные ресурсы (память).

1. Исключение общих подвыражений (оптимизация линейных участков)
2. Вычисления на этапе компиляции
3. Оптимизация булевых выражений
4. Вынесение инвариантных вычислений за пределы цикла

Исключение общих подвыражений

- Оптимизация линейных участков – это процедура, позволяющая не рассчитывать одно и то же выражение несколько раз, исключая общие подвыражения.
1. представить выражение в форме, пригодной для обнаружения общих подвыражений;
 2. определить эквивалентность двух и более подвыражений;
 3. исключить повторяющиеся;
 4. изменить команды так, чтобы учесть это исключение.

Исключение общих подвыражений

пусть выражение $A = c*d*(d*c+b)$ записано в виде тетрад:

- Упорядочим операнды в алфавитном порядке (там, где это возможно) [1];
- определим границы операторов и найдем общие подвыражения (это (1) и (2)) [2];
- затем исключим подвыражение (2).
- После чего заменим далее везде T2 на T1 [3].

```
1. * c d T1
2. * d c T2
3. + T2 b T3
4. * T1 T3 T4
5. = A T4
```

1

```
1. * c d T1
2. * c d T2
3. + T2 b T3
4. * T1 T3 T4
5. = A T4
```

2

```
1. * c d T1
2. + T1 b T3
3. * T1 T3 T4
4. = A T4
```

3

Вычисления на этапе компиляции

- Если в программе есть участки, в которых присутствуют подвыражения, состоящие из констант, то эти подвыражения можно просчитать еще на этапе компиляции.
- Например, вполне возможно заранее вычислить правую часть выражения вида:

$A := 1.5 * 2/3$

Оптимизация булевых выражений

- Метод основан на использовании свойств булевых выражений.

Вместо

if (a and b and c) then <операторы> endif

надо сгенерировать команды таким образом, чтобы исключались лишние проверки.

if not a then goto Label

if not b then goto Label

if not c then goto Label

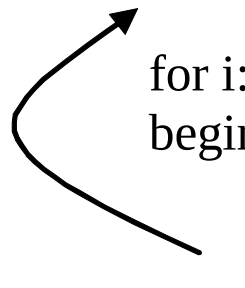
 <операторы>

Label: //метка перехода

Вынесение инвариантных вычислений за пределы цикла

Для реализации метода необходимо:

- распознать инвариант;
- определить место переноса;
- перенести инвариант.



```
for i:=1 to 10 do
begin
    .....
    a:=b+c; // инвариант
    .....
end
```

Проблемы, связанные с оптимизацией

- необходимо сопоставлять ожидаемый выигрыш от повышения эффективности объектной программы с дополнительными накладными расходами, связанными с **увеличением времени компиляции, надежности и сложности самого компилятора.**
- С оптимизаций зачастую связаны такие проблемы, как даже ухудшение кода, **сложность трассировки и отладки оптимизированных программ.**
- Побочные эффекты.

Побочные эффекты ОПТИМИЗАЦИИ

- Например, пусть мы работаем со стеком, используя функции *push()* для записи числа в стек и *pop()* для извлечения числа.

```
A := pop();           push(pop() || pop()),
```

```
B := pop();
```

```
C := A || B;
```

```
push(C);
```

- скорее всего оптимизирующий компилятор превратит это выражение в *push(pop())*, сочтя второй *pop()* лишним (он будет использовать логическую парадигму $A \parallel A \equiv A$).

Проверочная работа

1. Типы программ, включающие элементы трансляторов.
2. Назовите элементы компиляторов.
3. Перечислите этапы процесса компиляции.
4. Перечислите формы записи выражений и их особенности.
5. Определение алфавита языка.
6. Определение формального языка.
7. Определение и структура формальной грамматики.
8. Форма Бэкса-Науэра.
9. Грамматический разбор – цель, методы.
10. Восходящий и нисходящий грамматический разбор – особенности, отличия, преимущества.
11. Назовите основные типы грамматик по классификации Хомского. Какие из них используются в языках программирования.
12. Лексический анализ – определение, методы.
13. Синтаксический анализ – определение, методы.
14. Определение конечного автомата. Особенности при синтаксическом разборе.
15. Способы описания конечного автомата.
16. Определите автомат с магазинной памятью. В чем особенность его использования при грамматическом разборе?
17. Как построить автомат с магазинной памятью по синтаксическим диаграммам?